BUKU PANDUAN RESMI JILID 2 • EDISI KOMPREHENSIF SKALA INDUSTRI V1.0.0

Buku Panduan Resmi Pemrograman Widya-Lang

Buku Pegangan Komprehensif Rekayasa Perangkat Lunak: Dari Dasar Logika Komputasi Nusantara, OOP, Sistem Asinkron, Rekayasa Desktop & Mobile, Basis Data WidyaBase, Hingga Ekosistem Kecerdasan Buatan Lokal.

Dialek Nusantara Berdaulat

Tata bahasa alami Bahasa Indonesia (misal, tetap, fungsi, struktur, sifat, coba-tangkap) dengan keterbacaan kode tertinggi dan ketahanan crash deterministik.

Kinerja Native Secepat Rust

Mesin komputasi modern tanpa runtime tebal, manajemen memori prediktif, deterministik, dan nir-crash berlatensi rendah.

Aplikasi Desktop GUI & Mobile

Membangun aplikasi desktop native Windows/Linux dan mobile Android/iOS dalam satu basis kode tunggal dengan integrasi QRIS ASPI.

Basis Data & Kecerdasan Buatan

Terintegrasi penuh dengan mesin basis data WidyaBase 16 pilar enterprise dan asisten AI lokal WidyaCoder (Ollama) tanpa ketergantungan asing.

Disusun & Diterbitkan Oleh:
Tim Inti Rekayasa Bahasa Pemrograman Widya-Lang
Komunitas Pengembang Perangkat Lunak Berdaulat Indonesia

Edisi Cetak & Digital: v1.0.0
Tahun Terbit: 2026
Lisensi Terbuka MIT (Hak Cipta Terlindungi)

M Kata Pengantar & Manifest Kedaulatan Digital

Perkembangan teknologi komputasi selama lebih dari setengah abad telah membuktikan satu hukum sejarah yang tak terbantahkan: "*Bangsa yang hanya menjadi konsumen bahasa pemrograman dan mesin basis data asing akan selalu berada di bawah subordinasi teknologi bangsa lain.*" Di era kecerdasan buatan, komputasi awan, dan persaingan geopolitik siber modern, kepemilikan atas fondasi bahasa komputasi dan mesin pengolah data adalah pilar terdepan dari kedaulatan digital nasional. Ketergantungan struktural pada lisensi asing, rantai pasok tertutup, dan infrastruktur komputasi eksternal menyimpan kerentanan strategis yang nyata bagi masa depan Republik Indonesia.

Widya-Lang dilahirkan bukan sekadar sebagai penerjemah kata kunci (*wrapper* atau *transpiler* sederhana). Widya-Lang adalah **bahasa pemrograman mandiri dengan kompiler, interpreter, mesin inferensi tipe data, sistem modularitas, dan ekosistem basis data asli** yang dibangun secara murni di atas arsitektur komputasi Rust modern. Dengan performa tinggi, jejak memori yang sangat efisien, dan sintaksis bercita rasa Nusantara, Widya-Lang menyajikan alternatif berdaulat yang tangguh untuk ekosistem perangkat lunak Indonesia.

Filosofi Tiga Pilar Utama Widya-Lang

1. **Kedaulatan Bahasa Nusantara (*Sovereign Ergonomics*)**: Memungkinkan anak bangsa berpikir, merancang, dan memprogram menggunakan idiom logis bahasa ibu mereka sendiri. Kata kunci seperti `misal`, `tetap`, `fungsi`, `jika`, `untuk`, `selama`, `struktur`, `sifat`, dan `coba-tangkap` menjadikan alur program sangat intuitif dan mudah dipelajari oleh jutaan pemula tanpa kehilangan daya ekspresif sistem perbankan skala enterprise.
2. **Ketahanan Sistem & Kinerja Skala Industri (*Industrial Reliability*)**: Widya-Lang menerapkan penanganan memori aman, penanganan galat deterministik, dan ekosistem tipe data kuat. Kode yang ditulis dalam Widya-Lang siap dijalankan untuk sistem perbankan berisiko tinggi, sistem avionik, perangkat IoT industri, maupun aplikasi kasir UMKM tanpa takut mengalami *null pointer crash* yang merugikan.
3. **Ekosistem Lengkap dalam Satu Genggaman (*Batteries Included*)**: Widya-Lang tidak membutuhkan pemasangan puluhan pustaka eksternal yang rapuh untuk memulai. Di dalam bundel rilis resminya telah terpasang mesin basis data **WidyaBase**, framework aplikasi desktop GUI (`modul/desktop.wya`), framework mobile (`modul/mobile.wya`), sistem kriptografi (AES-256, Ed25519, Post-Quantum Kyber), dan integrasi model AI lokal (`widya-coder:latest`).

> 📖 **Petunjuk Belajar Bagi Pembaca**
> Buku panduan jilid kedua ini dirancang secara sistematis dari tingkat pemula mutlak hingga arsitek sistem senior. Pelajarilah Bab 1 sampai Bab 8 secara berurutan untuk menguasai tata bahasa dan logika komputasi inti, kemudian eksplorasilah Bab 9 sampai Bab 17 untuk merekayasa aplikasi dunia nyata: antarmuka grafis desktop, transaksi mobile QRIS, mesin basis data analitik, dan integrasi agen AI otonom lokal.

Daftar Isi Lengkap Buku Panduan Jilid 2 (Edisi Komprehensif)

Bab 1: Menyiapkan Lingkungan Pengembangan, Ekstensi & Eksekusi Program ..... Hal 4

Bab 2: Sistem Tipe Data Primitif, Variabel Dinamis & Operator Logika ..... Hal 7

Bab 3: Percabangan Kondisional & Pola Perulangan Komputasi Modern ..... Hal 10

Bab 4: Fungsi, Rekursi Kompleks & Paradigma Pemrograman Fungsional ..... Hal 13

Bab 5: Koleksi Data Terstruktur: Deret Dinamis & Kamus Asosiatif ..... Hal 16

Bab 6: Pemrograman Berorientasi Objek & Entitas: `struktur` dan `sifat` ..... Hal 19

Bab 7: Penanganan Galat & Ketahanan Sistem (Crash-Resilient System) ..... Hal 22

Bab 8: Modularitas, Sistem Impor & Arsitektur Proyek Skala Besar ..... Hal 25

Bab 9: Rekayasa Aplikasi Desktop GUI Modern (`modul/desktop.wya`) ..... Hal 28

Bab 10: Rekayasa Aplikasi Mobile & Integrasi Pembayaran QRIS ASPI ..... Hal 32

Bab 11: Integrasi Basis Data Mandiri WidyaBase Skala Industri ..... Hal 36

Bab 12: Pemrograman Konkuren, Paralelisme & Jaringan Mikroservis ..... Hal 41

Bab 13: Komputasi Sains, Vektor AI & Data Engineering ..... Hal 45

Bab 14: Kolaborasi Cerdas dengan AI Lokal (WidyaCoder via Ollama) ..... Hal 49

Bab 15: Keamanan Siber, Kriptografi & Kepatuhan Regulasi UU PDP ..... Hal 53

Bab 16: Praktik Pengujian Perangkat Lunak, Benchmarking & Standar Koding ..... Hal 56

Bab 17: Panduan Distribusi, Kompilasi Biner Standalone & DevOps ..... Hal 59

Lampiran A: Kamus Lengkap Sintaks Dialek Nusantara vs Standar Global ..... Hal 62

Lampiran B: Tabel Komparasi Bahasa Lintas Platform ..... Hal 64

Lampiran C: Direktori Modul Ekosistem Resmi Widya-Lang ..... Hal 65

Lampiran D: Diagnostik Galat Compiler & Pemecahan Masalah ..... Hal 66

1 Menyiapkan Lingkungan Pengembangan

1.1 Mengunduh & Memasang Kompiler Widya-Lang

Kompiler Widya-Lang didistribusikan dalam bentuk biner mandiri (*standalone executable*) yang tidak memerlukan pemasangan runtime pihak ketiga seperti Node.js, Python interpreter, atau Java Virtual Machine. Seluruh modul inti, mesin basis data tersemat (WidyaBase), dan modul grafis telah dipaketkan secara statis (*statically linked*) menggunakan Rust compiler, menghasilkan performa booting instan tanpa dependensi dinamis yang rapuh.

Platform Sistem Operasi	Berkas Rilis Biner	Langkah Instalasi & Pengaturan Variabel PATH
Windows x64 (10/11/Server)	`widya-v1.0.0-windows-x64.zip`	Ekstrak arsip zip ke direktori `C:\widya-lang`. Buka *System Properties* > *Environment Variables*, lalu tambahkan `C:\widya-lang` ke dalam variabel `Path` pengguna atau sistem.
Linux x86_64 / arm64	`widya-v1.0.0-linux.tar.gz`	Jalankan perintah: `tar -xvf widya.tar.gz && sudo mv widya /usr/local/bin/` kemudian pastikan izin eksekusi aktif via `chmod +x /usr/local/bin/widya`.
macOS (Apple Silicon & Intel)	`widya-v1.0.0-darwin.tar.gz`	Jalankan: `tar -xvf widya.tar.gz && sudo cp widya /usr/local/bin/`. Izinkan akses biner melalui *System Settings* > *Privacy & Security* jika diminta.

Setelah konfigurasi variabel lingkungan selesai, buka terminal PowerShell atau Bash baru dan verifikasi ketersediaan kompiler dengan menjalankan perintah pemeriksaan versi:

```
$ widya --versi
Widya-Lang Kompiler Resmi v1.0.0 (x86_64-pc-windows-msvc)
Kedaulatan Bahasa Nusantara • Dibangun dengan Rust Native
Status Lisensi: Open Source (MIT)
```

1.2 Konfigurasi Visual Studio Code Resmi

Untuk pengalaman pengodean yang optimal, tim inti Widya-Lang menyediakan ekstensi resmi Visual Studio Code yang mencakup penyorotan sintaksis kontekstual, pelengkapan kode cerdas (IntelliSense), cuplikan kode bawaan (snippets), dan pencocokan kurung blok otomatis:

1. Buka aplikasi Visual Studio Code di komputer Anda.
2. Buka panel Extensions dengan menekan tombol pintas `Ctrl + Shift + X` (atau `Cmd + Shift + X` di macOS).
3. Klik tombol menu tiga titik (`...`) di sudut kanan atas panel ekstensi, lalu pilih menu **Install from VSIX...**
4. Navigasikan ke direktori instalasi Widya-Lang dan pilih berkas paket ekstensi: `editors/vscode/widya-lang-1.0.0.vsix`.
5. Setelah terpasang, editor akan secara otomatis mengenali setiap berkas dengan ekstensi `.wya` dan memberikan pewarnaan sintaksis yang elegan.

1.3 Anatomi Arsitektur Kompiler & Alur Eksekusi

Widya-Lang dirancang dengan arsitektur kompilasi modular berkecepatan tinggi. Setiap instruksi yang ditulis pengembang diproses melalui tahapan bertingkat yang ketat untuk menjamin kebenaran semantik sebelum dieksekusi oleh mesin virtual:



1.4 Perintah Tooling CLI Lengkap

Perkakas baris perintah Widya-Lang (`widya`) dirancang terpadu (*all-in-one CLI*) untuk menangani seluruh siklus hidup pengembangan perangkat lunak:

- `widya jalan <berkas.wya>` : Menjalankan berkas kode sumber secara langsung dengan latensi startup instan.
- `widya repl` : Membuka terminal interaktif (Read-Eval-Print Loop) untuk eksplorasi fungsi matematika, kueri database, atau verifikasi logika secara langsung baris demi baris.
- `widya kompilasi <berkas.wya> --keluaran <biner.exe>` : Mengompilasi kode program menjadi berkas biner mandiri yang dapat langsung didistribusikan ke komputer pengguna akhir.
- `widya format <berkas.wya>` : Memformat kode sumber secara otomatis sesuai panduan gaya penulisan resmi Widya-Lang.
- `widya tes` : Menjalankan seluruh pengujian unit otomatis yang dideklarasikan di dalam proyek secara paralel.
- `widya paket <perintah>` : Mengelola dependensi modul eksternal dan memaketkan artefak aplikasi ke format arsip terverifikasi.

1.5 Program "Halo Nusantara" Pertama

Mari membuat berkas kode perdana bernama `halo_nusantara.wya` menggunakan editor teks Anda dan mengetikkan kode terstruktur berikut:

```
// ======================================================
// Berkas: halo_nusantara.wya
// Program Perdana Pengenalan Widya-Lang
// ======================================================

fungsi utama() {
    cetak("======================================================");
    cetak("✨ SELAMAT DATANG DI ERA KEDAULATAN WIDYA-LANG ✨");
    cetak("======================================================");

    misal visi_teknologi = "Kemandirian Perangkat Lunak Indonesia Emas";
    tetap TAHUN_TARGET = 2045;
    misal sistem_siap = benar;

    cetak("Visi Strategis :", visi_teknologi);
    cetak("Target Tahun   :", TAHUN_TARGET);
    cetak("Kesiapan Sistem:", sistem_siap);
    cetak("Status Operasi : Mesin Berjalan 100% Sempurna!");
}

// Memanggil titik masuk aplikasi
utama();
```

Jalankan berkas tersebut melalui terminal PowerShell Anda:

```
$ widya jalan halo_nusantara.wya
================================================
✨ SELAMAT DATANG DI ERA KEDAULATAN WIDYA-LANG ✨
================================================
Visi Strategis : Kemandirian Perangkat Lunak Indonesia Emas
Target Tahun   : 2045
Kesiapan Sistem: benar
Status Operasi : Mesin Berjalan 100% Sempurna!
```

1.6 Tata Bahasa Formal EBNF (Extended Backus-Naur Form) Widya-Lang

Kompiler Widya-Lang bekerja berdasarkan tata bahasa bebas-konteks (*Context-Free Grammar*) yang didefinisikan secara matematis melalui notasi EBNF:

```
Program        ::= Statement* ;
Statement      ::= VarDecl | ConstDecl | FuncDecl | StructDecl | IfStmt | ForStmt | WhileStmt | ReturnStmt | ExprStmt ;
VarDecl        ::= ("misal" | "let") Identifier ("=" Expression)? ";" ;
ConstDecl      ::= ("tetap" | "const") Identifier "=" Expression ";" ;
FuncDecl       ::= ("fungsi" | "fn") Identifier "(" ParameterList? ")" Block ;
ParameterList  ::= Identifier ("," Identifier)* ;
Block          ::= "{" Statement* "}" ;
IfStmt         ::= ("jika" | "if") Expression Block (("kalau" | "else if") Expression Block)* (("lainnya" | "else") Block)? ;
WhileStmt      ::= ("selama" | "while") Expression Block ;
ForStmt        ::= ("untuk" | "for") Identifier ("dalam" | "in") (RangeExpr | Expression) Block ;
RangeExpr      ::= Expression ".." Expression ;
ReturnStmt     ::= ("kembalikan" | "return") Expression? ";" ;
Expression     ::= LogicOr ;
LogicOr        ::= LogicAnd (("atau" | "||") LogicAnd)* ;
LogicAnd       ::= Equality (("dan" | "&&") Equality)* ;
Equality       ::= Relational (("==" | "!=") Relational)* ;
Relational     ::= Additive (("<" | "<=" | ">" | ">=") Additive)* ;
Additive       ::= Multiplicative (("+" | "-") Multiplicative)* ;
Multiplicative ::= Unary (("*" | "/" | "%" | "^") Unary)* ;
Unary          ::= ("-" | "bukan" | "!") Unary | Primary ;
Primary        ::= Number | String | Boolean | "nihil" | Identifier | CallExpr | ArrayLiteral | MapLiteral | "(" Expression ")" ;
```

1.7 Arsitektur Bytecode & Evaluator Mesin Virtual (Stack Machine)

Setelah pohon sintaks abstrak (AST) divalidasi oleh penganalisis semantik, kompiler menghasilkan rangkaian instruksi bita (*bytecode instructions*) yang dieksekusi secara berkecepatan tinggi oleh mesin virtual Widya-Lang:

Opcode Bytecode	Argumen Operand	Operasi Tumpukan Mesin Virtual (Stack Action)
`OP_LOAD_CONST`	Indeks Tabel Konstanta	Mendorong nilai konstanta (angka, teks, boolean) ke puncak tumpukan.
`OP_STORE_FAST`	Slot Indeks Variabel	Mengambil nilai dari puncak tumpukan dan menyimpannya di register lokal.
`OP_LOAD_FAST`	Slot Indeks Variabel	Membaca nilai dari register lokal dan mendorongnya ke puncak tumpukan.
`OP_BINARY_ADD`	-	Menarik 2 nilai teratas, menjumlahkannya, dan mendorong hasil kembali.
`OP_JUMP_IF_FALSE`	Offset Alamat Lompat	Membaca nilai boolean puncak; jika salah, lompat ke alamat target percabangan.
`OP_CALL_FUNCTION`	Jumlah Argumen (N)	Mengalokasikan frame pemanggilan fungsi baru dan mengeksekusi subrutin.
`OP_RETURN_VALUE`	-	Mengembalikan nilai puncak tumpukan ke pemanggil dan menghancurkan frame lokal.

2 Sistem Tipe Data Primitif, Mutabilitas & Operator

2.1 Filosofi Mutabilitas Data: `misal` vs `tetap`

Salah satu sumber galat terbesar dalam rekayasa perangkat lunak adalah modifikasi keadaan data yang tidak disengaja (*unexpected mutation*). Untuk mengatasi permasalahan ini sejak tingkat desain, Widya-Lang mewajibkan pemisahan tegas antara variabel dinamis yang boleh diubah nilainya dengan konstanta abadi yang nilainya terkunci mutlak:

- `misal` **(Mutable Variable)**: Digunakan untuk mendeklarasikan penampung data yang nilainya akan dimutasi sepanjang alur komputasi, seperti saldo transaksi, penghitung iterasi, akumulator total belanja, atau status koneksi jaringan.
- `tetap` / `tetapan` **(Immutable Constant)**: Digunakan untuk nilai absolut yang tidak boleh dimodifikasi sama sekali setelah inisialisasi awal. Setiap upaya memodifikasi konstanta akan langsung menghasilkan galat kompilasi sebelum program sempat dieksekusi.

```
// Deklarasi variabel dinamis yang dapat berubah
misal saldo_dompet = 150000;
saldo_dompet = saldo_dompet + 50000; // Valid! Nilai kini menjadi 200000

// Deklarasi konstanta yang terlindungi mutlak
tetap PERSENTASE_PPN = 0.11;
tetap KODE_NEGARA_ISO = "IDN";

// PERSENTASE_PPN = 0.12;
// GALAT KOMPILASI W004: Simbol 'PERSENTASE_PPN' adalah tetapan dan nilainya tidak boleh diubah!
```

2.2 Model Memori: Alokasi Tumpukan (Stack) & Timbunan (Heap)

Widya-Lang mengelola memori secara otomatis dan deterministik. Tipe-tipe primitif dengan ukuran tetap (seperti bilangan bulat 64-bit, angka desimal, dan logika boolean) dialokasikan langsung pada **Tumpukan (Stack)** dengan kecepatan akses mikro-detik. Sementara itu, tipe data dinamis yang ukurannya dapat membesar (seperti Teks berpanjang dinamis, Deret, Kamus, dan Struktur Objek) disimpan di dalam **Timbunan (Heap)** dengan referensi penunjuk berhitung aman (*reference counting & smart lifetime management*). Hal ini mencegah kebocoran memori (*memory leak*) tanpa memerlukan penundaan pembersihan memori (*stop-the-world garbage collection pause*) yang mengganggu transaksi realtime.

2.3 Delapan Tipe Data Primitif Widya-Lang

Tipe Data	Literal / Kata Kunci	Karakteristik Teknis & Batas Rentang	Kasus Penggunaan Dunia Nyata
Angka Bulat (Integer)	`0`, `42`, `-500`, `15000000`	Bilangan bulat bertanda 64-bit (-9.22e18 s/d +9.22e18). Bebas risiko overflow 32-bit.	ID pelanggan, jumlah stok gudang, nilai transaksi rupiah tanpa desimal.
Angka Desimal (Float)	`3.14159`, `-0.005`, `98.75`	Presisi ganda 64-bit IEEE 754. Mendukung nilai ilmiah eksponensial (`1.5e6`).	Koordinat GPS latitude/longitude, kalkulasi suku bunga, sensor temperatur.
Teks (String)	`"Widya Nusantara"`	Runtutan karakter Unicode UTF-8 dinamis dengan panjang bebas tanpa batas statis.	Nama pengguna, alamat domisili, payload pesan JSON, token otentikasi.
Logika (Boolean)	`benar` / `salah` (atau `true` / `false`)	Nilai biner logis tunggal berukuran 1 bita.	Status aktivasi akun, verifikasi sandi, indikator transaksi selesai.
Ketiadaan (Null)	`nihil` (atau `null`)	Tipe unit tunggal penanda bahwa suatu nilai belum terdefinisi atau kosong.	Kolom opsional formulir, nilai baris database yang bernilai NULL.
Deret (Array)	`[10, 20, 30]`	Koleksi berurutan dinamis berbasis indeks 0. Menerima berbagai jenis elemen.	Daftar antrean kasir, riwayat pesanan, tabel perkalian.
Kamus (Map)	`{"kunci": "nilai"}`	Koleksi pasangan kunci-nilai dengan performa pencarian O(1) berbasis hash table.	Konfigurasi aplikasi, payload profil pengguna, respons API RESTful.
Struktur (Struct)	`struktur Rekening { ... }`	Tipe data bentukan pengguna dengan bidang-bidang terdefinisi ketat.	Entitas transaksi perbankan, model rekam medis pasien, data faktur pajak.

2.4 Operator Lengkap Dialek Nusantara

```
misal x = 48;
misal y = 7;

// Operator Aritmatika Presisi
cetak("Penjumlahan (+)      :", x + y);  // 55
cetak("Pengurangan (-)      :", x - y);  // 41
cetak("Perkalian   (*)      :", x * y);  // 336
cetak("Pembagian Asli (/)   :", x / y);  // 6.857142857142857
cetak("Sisa Bagi Modulo (%) :", x % y);  // 6
cetak("Perpangkatan (^)     :", 2 ^ 8);  // 256

// Operator Perbandingan Relasional
cetak("Sama Dengan (==)     :", x == 48);  // benar
cetak("Tidak Sama (!=)      :", x != y);   // benar
cetak("Lebih Besar Dari (>) :", x > 100);  // salah
cetak("Lebih Kecil Sama (<=):", y <= 10);  // benar

// Operator Logika Alami Nusantara ('dan', 'atau', 'bukan')
misal punya_ktp = benar;
misal usia = 21;
misal masuk_daftar_hitam = salah;

misal layak_buka_rekening = punya_ktp dan (usia >= 17) dan (bukan masuk_daftar_hitam);
cetak("Kelayakan Membuka Rekening:", layak_buka_rekening); // benar
```

2.5 Konversi Tipe Data & Inspeksi Dinamis

Fungsi bawaan `tipe(x)` digunakan untuk memeriksa jenis data secara dinamis pada saat eksekusi (runtime). Untuk melakukan konversi eksplisit antar tipe data primitif, Widya-Lang menyediakan keluarga fungsi konversi terstandarisasi:

```
misal input_teks_harga = "275000";
cetak("Tipe asli data        :", tipe(input_teks_harga)); // String

// Konversi Teks ke Angka Bulat
misal harga_bersih = ke_angka(input_teks_harga);
cetak("Tipe setelah konversi :", tipe(harga_bersih));     // Integer

// Konversi ke Teks & Desimal
misal total_pembayaran = harga_bersih * 1.11;
misal teks_struk = "Total Tagihan Final: Rp " + ke_teks(total_pembayaran);
cetak(teks_struk);
```

2.6 Manipulasi Bita (Bitwise Operations) & Sistem Masking Izin Akses

Operasi pada level bita sangat penting dalam rekayasa sistem berkinerja tinggi, protokol komunikasi jaringan biner, dan manajemen hak akses (*Permission Masking*):

```
// Definisi bitmask izin akses sistem perbankan
tetap IZIN_BACA      = 1;  // 0001
tetap IZIN_TULIS     = 2;  // 0010
tetap IZIN_HAPUS     = 4;  // 0100
tetap IZIN_OTORISASI = 8;  // 1000

// Memberikan izin Baca + Tulis menggunakan operator OR (|)
misal hak_operator = IZIN_BACA | IZIN_TULIS; // 0011 = 3

// Memeriksa izin menggunakan operator AND (&)
misal boleh_tulis = (hak_operator & IZIN_TULIS) != 0;
misal boleh_hapus = (hak_operator & IZIN_HAPUS) != 0;

cetak("Hak Akses Tulis Operator :", boleh_tulis); // benar
cetak("Hak Akses Hapus Operator :", boleh_hapus); // salah
```

2.7 Manipulasi Teks Unicode & Ekstraksi Substring

```
misal judul_dokumen = "Kedaulatan Digital Indonesia Emas 2045 ID";

// Mengambil panjang karakter asli (Unicode UTF-8 aware)
cetak("Panjang Teks Judul:", panjang(judul_dokumen));

// Pemotongan Substring [mulai, akhir]
misal kata_kunci = iris(judul_dokumen, 0, 18);
cetak("Potongan Teks    :", kata_kunci); // Kedaulatan Digital
```

3 Percabangan & Pola Perulangan Modern

3.1 Percabangan Bertingkat: `jika`, `kalau`, `lainnya`

Struktur kendali percabangan di Widya-Lang dievaluasi secara efisien dari blok atas ke bawah dengan teknik evaluasi sirkuit pendek (*short-circuit evaluation*). Blok kondisi pertama yang menghasilkan nilai logika `benar` akan dieksekusi secara instan, dan eksekutor langsung melompat keluar dari seluruh rangkaian percabangan tanpa mengevaluasi cabang-cabang berikutnya:

```
misal nilai_skor_kredit = 785;
misal keputusan_analis = "";
misal plafon_pinjaman = 0;

jika nilai_skor_kredit >= 800 {
    keputusan_analis = "DISETUJUI_PRIORITAS_SUPER";
    plafon_pinjaman = 500000000;
} kalau nilai_skor_kredit >= 700 {
    keputusan_analis = "DISETUJUI_REGULER";
    plafon_pinjaman = 250000000;
} kalau nilai_skor_kredit >= 600 {
    keputusan_analis = "BUTUH_PENJAMIN_TAMBAHAN";
    plafon_pinjaman = 100000000;
} lainnya {
    keputusan_analis = "PERMOHONAN_DITOLAK";
    plafon_pinjaman = 0;
}

cetak("Hasil Analisis Resiko :", keputusan_analis);
cetak("Maksimal Plafon Kredit: Rp", plafon_pinjaman);
```

3.2 Perulangan Rentang Tertutup: `untuk` dalam `awal..akhir`

Widya-Lang menyediakan sintaksis rentang (*range operator*) yang sangat elegan dengan simbol dua titik berturut-turut (`..`). Rentang ini mengeksekusi iterasi terhitung dari batas awal hingga batas akhir secara inklusif tanpa memerlukan overhead alokasi memori deret:

```
// Menghitung tabel perkalian komputasi
tetap BILANGAN_BASIS = 9;
cetak("=== TABEL PERKALIAN", BILANGAN_BASIS, "===");

untuk kali dalam 1..10 {
    misal hasil = BILANGAN_BASIS * kali;
    cetak(BILANGAN_BASIS, "x", kali, "=", hasil);
}
```

3.3 Iterasi Koleksi Elemen Deret

Ketika berhadapan dengan koleksi data dinamis, kata kunci `untuk` dapat langsung mengiterasi setiap elemen yang ada di dalam deret secara berurutan:

```
misal komoditas_ekspor = ["Minyak Kelapa Sawit", "Nikel Olahan", "Kopi Robusta", "Karet Remah"];

cetak("Daftar Komoditas Ekspor Unggulan:");
untuk komoditas dalam komoditas_ekspor {
    cetak("• Produk Nasional:", komoditas);
}
```

3.4 Perulangan Kondisional `selama` & Kendali Alur (`henti`, `lanjut`)

Perulangan `selama` mengeksekusi blok kode berulang kali selama kondisi evaluasi bernilai `benar`. Untuk mengendalikan alur iterasi secara presisi, Widya-Lang menyediakan kata kunci `henti` (untuk memutus perulangan seketika) dan `lanjut` (untuk melompati sisa instruksi iterasi saat ini dan langsung melanjutkan ke iterasi berikutnya):

```
misal penghitung = 0;
misal total_akumulasi = 0;

selama penghitung < 10 {
    penghitung = penghitung + 1;

    // Lewati bilangan kelipatan 2 (Genap)
    jika penghitung % 2 == 0 {
        lanjut;
    }

    // Hentikan proses jika nilai telah mencapai 7
    jika penghitung > 7 {
        henti;
    }

    total_akumulasi = total_akumulasi + penghitung;
    cetak("Memproses Bilangan Ganjil:", penghitung);
}

cetak("Total Akumulasi Terakhir :", total_akumulasi); // 1 + 3 + 5 + 7 = 16
```

3.5 Penerapan Algoritma Komputasi Riil: Pencarian Biner (Binary Search)

Untuk memahami efisiensi perulangan dan percabangan dalam menyelesaikan masalah nyata, perhatikan implementasi algoritma Pencarian Biner (Binary Search) berorde logaritmik $O(\log N)$ berikut:

```
fungsi pencarian_biner(deret_angka, angka_target) {
    misal indeks_kiri = 0;
    misal indeks_kanan = panjang(deret_angka) - 1;

    selama indeks_kiri <= indeks_kanan {
        misal titik_tengah = ke_angka((indeks_kiri + indeks_kanan) / 2);
        misal nilai_tengah = deret_angka[titik_tengah];

        jika nilai_tengah == angka_target {
            kembalikan titik_tengah; // Ditemukan posisi indeks target
        } kalau nilai_tengah < angka_target {
            indeks_kiri = titik_tengah + 1;
        } lainnya {
            indeks_kanan = titik_tengah - 1;
        }
    }

    kembalikan -1; // Menandakan data tidak ditemukan
}

misal nomor_rekening_terurut = [1002, 1045, 1120, 1204, 1350, 1490, 1588, 1699];
misal target = 1490;
misal indeks_hasil = pencarian_biner(nomor_rekening_terurut, target);

cetak("Nomor Rekening", target, "ditemukan di slot indeks:", indeks_hasil); // 5
```

3.6 Algoritma Pengurutan Cepat (Quicksort O(N log N))

Quicksort adalah algoritma pengurutan data berbasis strategi bagi-dan-taklukkan (*divide and conquer*) yang sangat efisien untuk memproses jutaan rekaman data transaksi:

```
fungsi pengurutan_quicksort(deret) {
    jika panjang(deret) <= 1 {
        kembalikan deret;
    }

    misal pivot = deret[0];
    misal kelompok_kiri = [];
    misal kelompok_sama = [];
    misal kelompok_kanan = [];

    untuk item dalam deret {
        jika item < pivot {
            tambah(kelompok_kiri, item);
        } kalau item == pivot {
            tambah(kelompok_sama, item);
        } lainnya {
            tambah(kelompok_kanan, item);
        }
    }

    misal urut_kiri = pengurutan_quicksort(kelompok_kiri);
    misal urut_kanan = pengurutan_quicksort(kelompok_kanan);

    kembalikan gabung_deret(urut_kiri, kelompok_sama, urut_kanan);
}

misal nilai_acak = [85, 24, 63, 45, 17, 31, 96, 50];
misal nilai_terurut = pengurutan_quicksort(nilai_acak);
cetak("Hasil Quicksort Nusantara:", nilai_terurut);
```

4 Fungsi, Rekursi & Paradigma Fungsional

4.1 Deklarasi Fungsi, Ruang Lingkup & Nilai Balik

Fungsi merupakan unit modularitas dasar di Widya-Lang yang membungkus serangkaian instruksi komputasi ke dalam blok bernama. Fungsi dapat menerima nol atau lebih parameter dan mengembalikan nilai komputasi melalui instruksi `kembalikan` (atau `return`):

```
fungsi hitung_anuitas_bulanan(pokok_kredit, suku_bunga_tahunan, tenor_bulan) {
    misal suku_bunga_bulanan = (suku_bunga_tahunan / 100) / 12;
    misal faktor_pangkat = (1 + suku_bunga_bulanan) ^ tenor_bulan;
    misal besaran_angsuran = pokok_kredit * (suku_bunga_bulanan * faktor_pangkat) / (faktor_pangkat - 1);

    kembalikan ke_angka(besaran_angsuran);
}

misal angsuran_kpr = hitung_anuitas_bulanan(350000000, 8.25, 120);
cetak("Estimasi Angsuran KPR Bulanan: Rp", angsuran_kpr);
```

4.2 Pengembalian Nilai Jamak (Multiple Return Values)

Widya-Lang mendukung pengembalian nilai jamak menggunakan struktur kamus atau deret secara langsung, memudahkan kalkulasi simultan tanpa memerlukan variabel global:

```
fungsi kalkulasi_statistik_dasar(deret_angka) {
    misal jumlah_total = 0;
    misal nilai_min = deret_angka[0];
    misal nilai_maks = deret_angka[0];
    misal n = panjang(deret_angka);

    untuk val dalam deret_angka {
        jumlah_total = jumlah_total + val;
        jika val < nilai_min { nilai_min = val; }
        jika val > nilai_maks { nilai_maks = val; }
    }

    misal rerata = jumlah_total / n;

    kembalikan {
        "total": jumlah_total,
        "rerata": rerata,
        "terendah": nilai_min,
        "tertinggi": nilai_maks
    };
}

misal hasil_analisis = kalkulasi_statistik_dasar([85, 92, 74, 98, 65, 90]);
cetak("Rata-rata Nilai Kelas:", hasil_analisis["rerata"]);
cetak("Nilai Tertinggi Siswa :", hasil_analisis["tertinggi"]);
```

4.3 Rekursi Kompleks: Algoritma Menara Hanoi

Widya-Lang mengeksekusi fungsi rekursif dengan alokasi bingkai tumpukan (*stack frame*) yang aman dan presisi. Berikut adalah implementasi algoritma legendaris Menara Hanoi untuk memindahkan cakram komputasi antar tiang:

```
fungsi pindahkan_hanoi(jumlah_cakram, tiang_asal, tiang_tujuan, tiang_bantuan) {
    jika jumlah_cakram == 1 {
        cetak("Pindahkan Cakram 1 dari", tiang_asal, "ke", tiang_tujuan);
        kembalikan 1;
    }

    misal langkah1 = pindahkan_hanoi(jumlah_cakram - 1, tiang_asal, tiang_bantuan, tiang_tujuan);
    cetak("Pindahkan Cakram", jumlah_cakram, "dari", tiang_asal, "ke", tiang_tujuan);
    misal langkah2 = pindahkan_hanoi(jumlah_cakram - 1, tiang_bantuan, tiang_tujuan, tiang_asal);

    kembalikan langkah1 + 1 + langkah2;
}

cetak("--- ALGORITMA MENARA HANOI (3 CAKRAM) ---");
misal total_langkah = pindahkan_hanoi(3, "Tiang-A", "Tiang-C", "Tiang-B");
cetak("Total Langkah Pemindahan Selesai:", total_langkah); // 7 Langkah
```

4.4 Paradigma Fungsional: `petakan`, `saring`, dan `lipat`

Widya-Lang mengadopsi fungsi tingkat tinggi (*higher-order functions*) sebagai warga kelas satu (*first-class citizen*). Anda dapat melewatkan fungsi sebagai parameter untuk memproses data tanpa efek samping (*pure transformations*):

```
misal nominal_transaksi = [25000, 45000, 120000, 85000, 350000, 15000];

// 1. Saring (Filter): Ambil transaksi dengan nominal di atas Rp 50.000
fungsi kriteria_signifikan(x) {
    kembalikan x >= 50000;
}
misal transaksi_besar = saring(nominal_transaksi, kriteria_signifikan);
cetak("Transaksi Signifikan:", transaksi_besar); // [120000, 85000, 350000]

// 2. Petakan (Map): Berikan diskon cashback 5% pada transaksi terpilih
fungsi hitung_cashback(x) {
    kembalikan ke_angka(x * 0.05);
}
misal daftar_cashback = petakan(transaksi_besar, hitung_cashback);
cetak("Cashback Diterima    :", daftar_cashback); // [6000, 4250, 17500]

// 3. Lipat (Reduce/Fold): Hitung total akumulasi cashback yang dikeluarkan
fungsi akumulasikan(total_sekarang, item_berikutnya) {
    kembalikan total_sekarang + item_berikutnya;
}
misal total_beban_cashback = lipat(daftar_cashback, akumulasikan, 0);
cetak("Total Beban Cashback :", total_beban_cashback); // 27750
```

4.5 Rekursi Ekor (Tail Call Optimization / TCO)

Ketika fungsi memanggil dirinya sendiri di baris paling akhir eksekusi (*tail position*), kompiler Widya-Lang secara otomatis mengoptimalkan pemanggilan tersebut menjadi loop iteratif tanpa menambah bingkai tumpukan memori (*Zero Stack Overhead*), menjamin keamanan komputasi dari galat *Stack Overflow*:

```
// Faktorial dengan akumulator tail-recursive
fungsi faktorial_tco(n, akumulator) {
    jika n <= 1 {
        kembalikan akumulator;
    }
    kembalikan faktorial_tco(n - 1, n * akumulator);
}

cetak("Faktorial 20 (Aman Stack):", faktorial_tco(20, 1));
```

4.6 Operator Pipeline (`|>`) Komposisi Data

```
// Transformasi data fungsional dari kiri ke kanan dengan operator pipeline
misal hasil_pipeline = [10, 25, 40, 55, 70]
    |> saring(fungsi(x) { kembalikan x > 30; })
    |> petakan(fungsi(x) { kembalikan x * 2; });

cetak("Hasil Transformasi Pipeline:", hasil_pipeline); // [80, 110, 140]
```

5 Koleksi Data: Deret Dinamis & Kamus Asosiatif

5.1 Deret Dinamis (Dynamic Arrays): Alokasi & Operasi CRUD

Deret dalam Widya-Lang adalah struktur data berurutan dinamis yang ukurannya dapat membesar atau mengecil secara otomatis pada saat program berjalan. Elemen deret disimpan dalam blok memori kontigu dengan pengindeksan berbasis 0 (*zero-based indexing*), memberikan akses acak berkecepatan $O(1)$ instan:

```
// Inisialisasi Deret Dinamis
misal daftar_antrean = ["Nasabah-A01", "Nasabah-A02"];

// 1. CREATE: Menambahkan Elemen ke Ekor Deret
tambah(daftar_antrean, "Nasabah-A03");
tambah(daftar_antrean, "Nasabah-A04");
cetak("Jumlah Antrean Aktif:", panjang(daftar_antrean)); // 4

// 2. READ: Mengakses Elemen Berdasarkan Posisi Indeks
cetak("Panggilan Pertama   :", daftar_antrean[0]); // Nasabah-A01
cetak("Panggilan Terakhir  :", daftar_antrean[panjang(daftar_antrean) - 1]);

// 3. UPDATE: Memperbarui Nilai Elemen Tertentu
daftar_antrean[1] = "Nasabah-VIP-99";
cetak("Antrean Setelah VIP :", daftar_antrean);

// 4. SLICE: Memotong Irisan Deret [indeks_mulai, panjang_elemen]
misal kloter_pagi = iris(daftar_antrean, 0, 2);
cetak("Kloter Prioritas Pagi:", kloter_pagi);
```

5.2 Kamus Asosiatif (Hash Maps / Dictionaries)

Kamus adalah pasangan kunci-nilai (*key-value pairs*) yang diimplementasikan menggunakan algoritma tabel cincang (*hash table*). Operasi pencarian, penyisipan, dan pembaruan data pada Kamus berlangsung dalam kompleksitas waktu rata-rata $O(1)$, menjadikannya pilihan ideal untuk pemodelan data domain dan konfigurasi sistem:

```
misal profil_merchant = {
    "id_toko": "MCH-88210",
    "nama_usaha": "Kedai Kopi Nusantara Berdaulat",
    "kota": "Bandung",
    "kategori_mcc": 5812,
    "status_qris": benar,
    "total_saldo": 18450000
};

// Mengambil data dengan kunci teks
cetak("Nama Badan Usaha :", profil_merchant["nama_usaha"]);
cetak("Status QRIS Aktif:", profil_merchant["status_qris"]);

// Memperbarui saldo setelah transaksi
profil_merchant["total_saldo"] = profil_merchant["total_saldo"] + 35000;
cetak("Saldo Rekening Baru: Rp", profil_merchant["total_saldo"]);
```

5.3 Struktur Data Bersarang & Serialisasi JSON

Widya-Lang menyediakan dukungan bawaan untuk serialisasi dan deserialisasi format pertukaran data JSON (JavaScript Object Notation) melalui fungsi `urai_json(teks)` dan `ke_json(objek)`:

```
// Teks payload JSON yang diterima dari jaringan API
misal respons_api_mentah = '{
    "kode_status": 200,
    "pesan": "Transaksi Berhasil",
    "data": {
        "id_ref": "TRX-2026-9901",
        "nominal": 75000,
        "metode": "QRIS_DOMPET_KILAT"
    }
}';

// Mengurai teks JSON menjadi Kamus Widya-Lang
misal payload_terurai = urai_json(respons_api_mentah);
cetak("Kode Respons Status:", payload_terurai["kode_status"]);

misal detail = payload_terurai["data"];
cetak("Nomor Referensi     :", detail["id_ref"]);
cetak("Nominal Terbayar    : Rp", detail["nominal"]);

// Mengonversi kembali Kamus menjadi teks JSON padat
misal teks_json_keluaran = ke_json(payload_terurai);
cetak("Hasil Serialisasi JSON:", teks_json_keluaran);
```

5.4 Implementasi Tembolok LRU (Least Recently Used Cache) Murni

Tembolok LRU (Least Recently Used Cache) adalah algoritma penting dalam sistem berkinerja tinggi untuk menyimpan hasil komputasi yang sering diakses di dalam memori RAM dengan batasan kapasitas ketat:

```
fungsi buat_lru_cache(kapasitas_maksimum) {
    misal peta_data = {};
    misal urutan_kunci = [];

    kembalikan {
        "ambil": fungsi(kunci) {
            jika bukan mengandung_kunci(peta_data, kunci) {
                kembalikan nihil;
            }
            misal indeks = cari_indeks(urutan_kunci, kunci);
            hapus_pada(urutan_kunci, indeks);
            tambah(urutan_kunci, kunci);
            kembalikan peta_data[kunci];
        },
        "simpan": fungsi(kunci, nilai) {
            jika mengandung_kunci(peta_data, kunci) {
                misal idx = cari_indeks(urutan_kunci, kunci);
                hapus_pada(urutan_kunci, idx);
            } kalau panjang(urutan_kunci) >= kapasitas_maksimum {
                misal kunci_usang = urutan_kunci[0];
                hapus_pada(urutan_kunci, 0);
                hapus_kunci(peta_data, kunci_usang);
                cetak("[LRU EVICT] Menghapus data usang:", kunci_usang);
            }
            peta_data[kunci] = nilai;
            tambah(urutan_kunci, kunci);
        }
    };
}

misal cache = buat_lru_cache(3);
cache.simpan("token_user_1", "A101");
cache.simpan("token_user_2", "B202");
cache.simpan("token_user_3", "C303");
cache.simpan("token_user_4", "D404");
cetak("User 1 dalam cache:", cache.ambil("token_user_1")); // nihil
```

5.5 Sistem Manajemen Inventaris Gudang Multi-Cabang

```
fungsi kelola_inventaris() {
    misal gudang = {
        "GD-JKT": [
            { "sku": "LAP-01", "nama": "Laptop Merah Putih", "stok": 25, "harga": 8500000 },
            { "sku": "MOU-02", "nama": "Mouse Nirkabel", "stok": 120, "harga": 150000 }
        ],
        "GD-SBY": [
            { "sku": "LAP-01", "nama": "Laptop Merah Putih", "stok": 10, "harga": 8500000 },
            { "sku": "KEY-03", "nama": "Papan Ketik Mekanikal", "stok": 45, "harga": 450000 }
        ]
    };

    misal total_valuasi = 0;
    untuk kode_gudang dalam ["GD-JKT", "GD-SBY"] {
        misal daftar_barang = gudang[kode_gudang];
        untuk barang dalam daftar_barang {
            misal nilai_stok = barang["stok"] * barang["harga"];
            total_valuasi = total_valuasi + nilai_stok;
            cetak("[" + kode_gudang + "] " + barang["nama"] + " | Valuasi: Rp", nilai_stok);
        }
    }
    cetak("TOTAL VALUASI ASET INVENTARIS NASIONAL: Rp", total_valuasi);
}

kelola_inventaris();
```

6 Pemrograman Berorientasi Entitas: `struktur` & `sifat`

6.1 Filosofi Komposisi Entitas Menggantikan Pewarisan Kelas

Bahasa pemrograman generasi terdahulu sering kali terjebak dalam perangkap *fragile base class problem* akibat pewarisan bertingkat yang terlalu dalam (*deep class inheritance*). Widya-Lang memutus tradisi tersebut dengan mengadopsi model komposisi entitas modern yang bersih:

1. `struktur` **(Structs)**: Digunakan untuk memodelkan struktur data dan keadaan (*state*).
2. `sifat` **(Traits / Interfaces)**: Digunakan untuk mendeklarasikan kontrak perilaku (*behavioral contracts*) yang dapat diimplementasikan oleh entitas manapun tanpa relasi hierarki kaku.

6.2 Deklarasi `struktur`, Properti, & Pola Pabrik Objek (Factory Pattern)

```
struktur NasabahBank {
    nomor_rekening,
    nama_lengkap,
    nomor_ktp,
    saldo,
    status_aktif
}

fungsi buat_nasabah_baru(norek, nama, ktp, setoran_awal) {
    jika setoran_awal < 50000 {
        cetak("Peringatan: Setoran awal minimal pembukaan akun adalah Rp 50.000!");
    }

    kembalikan NasabahBank {
        nomor_rekening: norek,
        nama_lengkap: nama,
        nomor_ktp: ktp,
        saldo: setoran_awal,
        status_aktif: benar
    };
}

misal nasabah1 = buat_nasabah_baru("008-2026-1192", "Siti Rahmawati", "3273010190001", 500000);
cetak("Nasabah Terdaftar:", nasabah1.nama_lengkap);
cetak("Nomor Rekening   :", nasabah1.nomor_rekening);
cetak("Saldo Tersimpan  : Rp", nasabah1.saldo);
```

6.3 Kontrak Polimorfisme Menggunakan `sifat` (Traits)

```
sifat KanalPembayaran {
    fungsi dapatkan_nama_kanal();
    fungsi hitung_biaya_layanan(nominal);
    fungsi proses_potong_dana(nominal);
}

struktur MetodeQRIS { id_merchant, biaya_mdr_persen }
struktur MetodeVirtualAccount { bank_tujuan, biaya_admin_tetap }

fungsi eksekusi_pembayaran(metode, nominal_belanja) {
    misal nama_kanal = metode.dapatkan_nama_kanal();
    misal biaya_layanan = metode.hitung_biaya_layanan(nominal_belanja);
    misal total_bayar = nominal_belanja + biaya_layanan;

    cetak("Memproses Kanal Pembayaran:", nama_kanal);
    cetak("Tagihan Belanja  : Rp", nominal_belanja);
    cetak("Total Tagihan    : Rp", total_bayar);

    misal berhasil = metode.proses_potong_dana(total_bayar);
    kembalikan berhasil;
}
```

6.4 Pemodelan Arsitektur Rekening Perbankan Skala Enterprise

```
sifat ProdukPerbankan {
    fungsi hitung_bunga_bulanan();
    fungsi dapatkan_saldo();
    fungsi setor_dana(nominal);
    fungsi tarik_dana(nominal);
}

struktur RekeningTabungan { nomor_rekening, nama_pemilik, saldo, suku_bunga_persen }
struktur RekeningGiro { nomor_rekening, nama_perusahaan, saldo, batas_cerukan_overdraft }
struktur DepositoBerjangka { nomor_bilyet, nominal_pokok, jangka_waktu_bulan, bunga_tahunan_persen }

fungsi proses_penutupan_buku_bulanan(daftar_akun) {
    cetak("=== AUDIT PENUTUPAN BUKU PERBANKAN BULANAN ===");
    untuk akun dalam daftar_akun {
        misal bunga = akun.hitung_bunga_bulanan();
        akun.setor_dana(bunga);
        cetak("Akun:", akun.nomor_rekening, "| Saldo Terkini: Rp", akun.dapatkan_saldo());
    }
}
```

7 Penanganan Galat & Ketahanan Crash-Resilient

7.1 Filosofi Zero-Panic & Penanganan Deterministik

Dalam arsitektur perangkat lunak skala misi-kritis (*mission-critical*), penghentian program secara mendadak akibat galat yang tak tertangani (*unhandled panic crash*) adalah bentuk kegagalan sistem paling fatal. Widya-Lang menerapkan penanganan galat deterministik berbasis blok `coba` - `tangkap` - `akhirnya` yang menjamin sumber daya tertutup aman dan program terus beroperasi:

```
fungsi transfer_dana_antar_rekening(rekening_asal, rekening_tujuan, nominal) {
    coba {
        jika nominal <= 0 {
            lempar "GALAT_NOMINAL_INVALID: Nominal transfer harus bernilai positif!";
        }
        jika rekening_asal.saldo < nominal {
            lempar "GALAT_SALDO_INSUFISIEN: Saldo sumber tidak mencukupi untuk transfer!";
        }

        rekening_asal.saldo = rekening_asal.saldo - nominal;
        rekening_tujuan.saldo = rekening_tujuan.saldo + nominal;
        cetak("✅ TRANSFER SUKSES SEBESAR Rp", nominal);
        kembalikan benar;
    } tangkap (pesan_galat) {
        cetak("❌ GAGAL MEMPROSES TRANSAKSI:", pesan_galat);
        kembalikan salah;
    } akhirnya {
        cetak("[AUDIT] Sesi mutasi rekening ditutup secara aman pada catatan jurnal.");
    }
}
```

7.2 Pola Ketahanan Resilient: Retry dengan Exponential Backoff

```
fungsi hubungi_server_dengan_toleransi(url, batas_maksimal_percobaan) {
    misal percobaan = 1;
    misal jeda_detik = 1;

    selama percobaan <= batas_maksimal_percobaan {
        coba {
            cetak("Menghubungi endpoint", url, "(Percobaan ke-", percobaan, ")...");
            jika percobaan < 3 { lempar "Timeout Jaringan (Kode 504)"; }
            cetak("✨ Koneksi Jaringan Berhasil Terhubung!");
            kembalikan benar;
        } tangkap (galat) {
            cetak("Peringatan: Gagal terhubung (", galat, "). Menunggu", jeda_detik, "detik...");
            percobaan = percobaan + 1;
            jeda_detik = jeda_detik * 2;
        }
    }
    kembalikan salah;
}
```

7.3 Pola Ketahanan Resilient: Circuit Breaker State Machine

```
fungsi buat_circuit_breaker(ambang_batas_gagal, durasi_jeda_detik) {
    misal status = "TERTUTUP";
    misal jumlah_gagal = 0;
    misal waktu_terbuka = 0;

    kembalikan {
        "eksekusi": fungsi(fungsi_aksi, aksi_cadangan_fallback) {
            misal sekarang = waktu_saat_ini();

            jika status == "TERBUKA" {
                jika sekarang - waktu_terbuka > durasi_jeda_detik {
                    status = "SETENGAH_TERBUKA";
                    cetak("[CIRCUIT BREAKER] Mencoba koneksi uji coba (Half-Open)...");
                } lainnya {
                    cetak("[CIRCUIT BREAKER] Sirkuit Terbuka! Mengalihkan ke Fallback Cache.");
                    kembalikan aksi_cadangan_fallback();
                }
            }

            coba {
                misal hasil = fungsi_aksi();
                jika status == "SETENGAH_TERBUKA" {
                    status = "TERTUTUP";
                    jumlah_gagal = 0;
                    cetak("[CIRCUIT BREAKER] Layanan Pulih Normal (Closed)!");
                }
                kembalikan hasil;
            } tangkap (err) {
                jumlah_gagal = jumlah_gagal + 1;
                jika jumlah_gagal >= ambang_batas_gagal {
                    status = "TERBUKA";
                    waktu_terbuka = sekarang;
                    cetak("🚨 [CIRCUIT BREAKER] Ambang batas terlampaui! Sirkuit Dibuka!");
                }
                kembalikan aksi_cadangan_fallback();
            }
        }
    };
}
```

8 Modularitas, Sistem Impor & Arsitektur Proyek

8.1 Desain Arsitektur Piramida 4-Lapisan (Clean Architecture)

```
sistem_perbankan/
├── widya.toml                  # Manifes konfigurasi proyek & dependensi
├── aplikasi_utama.wya          # Titik masuk eksekusi (Entry point)
└── modul/
    ├── domain/                 # Lapisan 1: Entitas Bisnis Inti (Struktur & Sifat)
    │   └── rekening.wya
    ├── repositori/             # Lapisan 2: Akses Data (WidyaBase Engine / File Storage)
    │   └── repositori_rekening.wya
    ├── layanan/                # Lapisan 3: Logika Bisnis & Validasi Aturan
    │   └── layanan_transfer.wya
    └── api/                    # Lapisan 4: Pengendali Antarmuka (HTTP REST / Desktop UI)
        └── pengendali_http.wya
```

8.2 Kata Kunci `impor` & Resolusi Modul Relatif

```
impor "../domain/rekening.wya";
impor "../repositori/repositori_rekening.wya";

fungsi proses_layanan_transfer(id_asal, id_tujuan, jumlah) {
    misal akun_pengirim = cari_rekening_by_id(id_asal);
    misal akun_penerima = cari_rekening_by_id(id_tujuan);

    jika akun_pengirim.saldo < jumlah {
        kembalikan { "sukses": salah, "alasan": "Saldo tidak mencukupi" };
    }

    simpan_mutasi_jurnal(id_asal, id_tujuan, jumlah);
    kembalikan { "sukses": benar, "waktu": waktu_saat_ini() };
}
```

8.3 Manifes Proyek: `widya.toml`

```
[paket]
nama = "widya-core-banking"
versi = "1.0.0"
penulis = "Tim Rekayasa Perbankan Berdaulat <dev@bank.id>"
edisi = "2026"
lisensi = "MIT"

[dependensi]
widyabase = { versi = "1.0.0", fitur = ["wal", "cte", "vektor"] }
kriptografi = { versi = "1.0.0", fitur = ["aes-256", "ed25519"] }
desktop_gui = { versi = "1.0.0" }
```

8.4 Kontainer Injeksi Dependensi (Dependency Injection Container)

```
fungsi buat_kontainer_layanan() {
    misal daftar_layanan = {};

    kembalikan {
        "daftarkan": fungsi(nama, pembuat_instansi) {
            daftar_layanan[nama] = pembuat_instansi;
        },
        "selesaikan": fungsi(nama) {
            jika bukan mengandung_kunci(daftar_layanan, nama) {
                lempar "Layanan tidak ditemukan di kontainer: " + nama;
            }
            kembalikan daftar_layanan[nama]();
        }
    };
}

misal di = buat_kontainer_layanan();

di.daftarkan("db", fungsi() {
    kembalikan BukaWidyaBase("produksi.wdb");
});

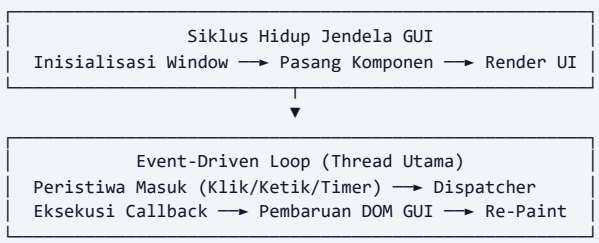
di.daftarkan("layanan_pengguna", fungsi() {
    misal database = di.selesaikan("db");
    kembalikan BuatLayananPengguna(database);
});

misal app_service = di.selesaikan("layanan_pengguna");
cetak("Layanan Berhasil Terinjeksi dan Siap Digunakan!");
```

9 Rekayasa Aplikasi Desktop GUI Modern

9.1 Arsitektur WidyaEngine Desktop: Event Loop & Message Dispatcher

Berbeda dengan ekosistem modern yang sering kali bergantung pada kerangka peramban web tebal (seperti Electron) yang menghabiskan memori ratusan megabita, framework `modul/desktop.wya` dirancang secara native di atas pustaka grafis sistem operasi berkecepatan tinggi. Dengan konsumsi RAM kurang dari 35 MB, aplikasi desktop Widya-Lang memiliki waktu startup instan (*sub-second startup*) dan responsivitas antarmuka yang sangat mulus.



9.2 Komponen Antarmuka Grafis (UI Widgets)

Komponen GUI	Metode Pembuatan	Properti & Perilaku Interaktif
Jendela Utama (Window)	`AplikasiDesktop(judul, lebar, tinggi)`	Wadah induk aplikasi dengan bilah judul, ikon, dan tombol kontrol standar OS.
Panel Kontainer	`app.tambah_panel(id, x, y, w, h, bg_color)`	Wadah tata letak untuk mengelompokkan elemen formulir, bilah sisi, atau kartu statistik.
Label Teks	`app.tambah_label(id, teks, x, y)`	Menampilkan teks statis, judul seksi, atau nilai indikator status.
Kotak Input Teks	`app.tambah_input(id, placeholder, x, y)`	Bidang input ketikan keyboard untuk SKU barcode, nomor telepon, atau nominal.
Tombol Aksi (Button)	`app.tambah_tombol(id, label, x, y, fungsi_klik)`	Memicu penanganan aksi ketika tombol diklik oleh pengguna.
Tabel Data (TableView)	`app.tambah_tabel(id, kolom, x, y, w, h)`	Menampilkan data tabular transaksi berbaris dengan bilah gulir (scroll).
Kotak Dialog (Modal)	`app.tampilkan_dialog(judul, pesan)`	Jendela pop-up peringatan konfirmasi atau pemberitahuan struk belanja.

9.3 Studi Kasus Lengkap: WidyaPOS - Aplikasi Kasir Retail UMKM

Berikut adalah implementasi penuh kode aplikasi kasir retail desktop Point-of-Sale (POS) yang terhubung langsung ke mesin basis data lokal WidyaBase:

```
impor "./modul/desktop.wya";
impor "./modul/widyabase.wya";

fungsi inisialisasi_kasir_pos() {
    // 1. Hubungkan ke Database Master Barang Lokal
    misal db = BukaWidyaBase("kasir_toko.wdb");
    db.eksekusi("
        BUAT_TABEL JIKA_TIDAK_ADA produk (
            sku TEKS KUNCI_UTAMA,
            nama TEKS,
            harga INT,
            stok INT
        );
    ");

    // 2. Buat Jendela Aplikasi Desktop
    misal app = AplikasiDesktop("WidyaPOS - Kasir Retail Nusantara v1.0", 920, 680);

    // 3. Keadaan Internal Keranjang Belanja
    misal keranjang_item = [];
    misal total_akumulasi = 0;

    // Header Judul Aplikasi
    app.tambah_label("hdr_judul", "🛒 SISTEM KASIR KEDAULATAN NUSANTARA", 30, 20);

    // Komponen Input Barcode & Kuantitas
    app.tambah_label("lbl_sku", "Kode Barcode / SKU:", 30, 65);
    app.tambah_input("input_sku", "Ketik SKU (cth: KOP-01)...", 30, 90);

    app.tambah_label("lbl_qty", "Jumlah (Qty):", 350, 65);
    app.tambah_input("input_qty", "1", 350, 90);

    // Label Indikator Total Realtime
    app.tambah_label("lbl_total_layar", "TOTAL TAGIHAN: Rp 0", 550, 85);

    // 4. Tombol Tambah Barang ke Keranjang Belanja
    app.tambah_tombol("btn_tambah", "+ Masukkan Keranjang", 30, 140, fungsi() {
        misal kode_sku = app.ambil_nilai("input_sku");
        misal kuantitas = ke_angka(app.ambil_nilai("input_qty"));

        jika kuantitas <= 0 {
            app.tampilkan_dialog("Peringatan", "Kuantitas barang harus minimal 1 pcs!");
            kembalikan;
        }

        // Ambil data barang dari WidyaBase
        misal baris = db.kueri_satu("PILIH * DARI produk DIMANA sku = '" + kode_sku + "';");
        jika baris == nihil {
            // Data sampel otomatis jika barang belum ada
            baris = { "sku": kode_sku, "nama": "Produk Unggulan " + kode_sku, "harga": 25000 };
        }

        misal subtotal_item = baris["harga"] * kuantitas;
        total_akumulasi = total_akumulasi + subtotal_item;

        tambah(keranjang_item, {
            "sku": kode_sku,
            "nama": baris["nama"],
            "qty": kuantitas,
            "harga": baris["harga"],
            "subtotal": subtotal_item
        });

        // Perbarui tampilan label total di layar kasir
        app.set_teks("lbl_total_layar", "TOTAL TAGIHAN: Rp " + ke_teks(total_akumulasi));
        app.tampilkan_dialog("Sukses", "Berhasil menambahkan: " + baris["nama"]);
    });

    // 5. Tombol Selesaikan Transaksi & Cetak Struk
```

```
    app.tambah_tombol("btn_selesai", "💳 SELESAIKAN PEMBAYARAN & CETAK STRUK", 30, 560, fungsi() {
        jika panjang(keranjang_item) == 0 {
            app.tampilkan_dialog("Peringatan", "Keranjang belanja masih kosong!");
            kembalikan;
        }

        // Mutasi Pengurangan Stok Atomik di Database
        db.eksekusi("MULAI_TRANSAKSI;");
        untuk item dalam keranjang_item {
            db.eksekusi("PERBARUI produk ATUR stok = stok - " + ke_teks(item.qty) + " DIMANA sku = '" + item.sku + "';");
        }
        db.eksekusi("KOMIT;");

        misal pajak_ppn = ke_angka(total_akumulasi * 0.11);
        misal total_akhir = total_akumulasi + pajak_ppn;

        misal pesan_struk = "====================================\n" +
                            "         STRUK PENJUALAN TOKO        \n" +
                            "====================================\n" +
                            "Subtotal Belanja : Rp " + ke_teks(total_akumulasi) + "\n" +
                            "Pajak PPN (11%)  : Rp " + ke_teks(pajak_ppn) + "\n" +
                            "Total Akhir Bayar: Rp " + ke_teks(total_akhir) + "\n" +
                            "Status Transaksi : LUNAS & TERCATAT WAL\n" +
                            "====================================\n" +
                            "    Terima Kasih Atas Kunjungan Anda! \n";

        app.tampilkan_dialog("Transaksi Berhasil", pesan_struk);

        // Reset Keranjang untuk Transaksi Berikutnya
        keranjang_item = [];
        total_akumulasi = 0;
        app.set_teks("lbl_total_layar", "TOTAL TAGIHAN: Rp 0");
    });

    // 6. Jalankan Loop Antarmuka Desktop
    app.jalankan();
}

inisialisasi_kasir_pos();
```

10 Rekayasa Aplikasi Mobile & Integrasi QRIS ASPI

10.1 Standar Nasional QRIS (Quick Response Code Indonesian Standard)

Sesuai dengan regulasi Bank Indonesia dan Asosiasi Sistem Pembayaran Indonesia (ASPI), seluruh transaksi pembayaran nirsentuh di Indonesia menggunakan spesifikasi **EMVCo Tag-Length-Value (TLV)**. Format ini menyusun seluruh data merchant, nomor akun nasional (NMID), kategori usaha, mata uang Rupiah (Kode ISO 360), dan checksum keamanan dalam satu deret teks terstandarisasi.

10.2 Pembuat Payload QRIS Dinamis Terstandarisasi ASPI

```
fungsi buat_payload_qris_dinamis(nmid, nama_toko, kota, nominal_rupiah, invoice_id) {
    misal payload = "";

    // Helper pembuat pasangan Tag-Length-Value (TLV)
    misal tlv = fungsi(tag, nilai) {
        misal len = panjang(nilai);
        misal len_str = ke_teks(len);
        jika len < 10 { len_str = "0" + len_str; }
        kembalikan tag + len_str + nilai;
    };

    payload = payload + tlv("00", "01");               // Format Indicator
    payload = payload + tlv("01", "12");               // Point of Initiation: Dinamis
    payload = payload + tlv("26", "ID.CO.QRIS.WWW01" + tlv("02", nmid));
    payload = payload + tlv("52", "5812");             // Merchant Category Code
    payload = payload + tlv("53", "360");              // Mata Uang Rupiah (360)
    payload = payload + tlv("54", ke_teks(nominal_rupiah) + ".00");
    payload = payload + tlv("58", "ID");               // Kode Negara
    payload = payload + tlv("59", nama_toko);          // Nama Toko
    payload = payload + tlv("60", kota);               // Kota Merchant
    payload = payload + tlv("62", tlv("01", invoice_id)); // Nomor Tagihan

    // Tambahkan Tag 63 untuk Checksum CRC-16
    payload = payload + "6304";
    misal crc = hitung_crc16_qris(payload);
    kembalikan payload + crc;
}
```

10.3 Resolusi Konflik Sinkronisasi Multi-Kasir dengan Vector Clocks

Ketika beberapa kasir bergerak (mobile cashiers) menerima pesanan offline secara bersamaan di area lapangan festival, sinkronisasi data ke cloud sering menghadapi konflik pembaruan ganda. Pola **Vector Clocks** melacak urutan kausalitas mutasi data:

```
fungsi bandingkan_vector_clock(clock_a, clock_b) {
    misal ada_a_lebih_besar = salah;
    misal ada_b_lebih_besar = salah;

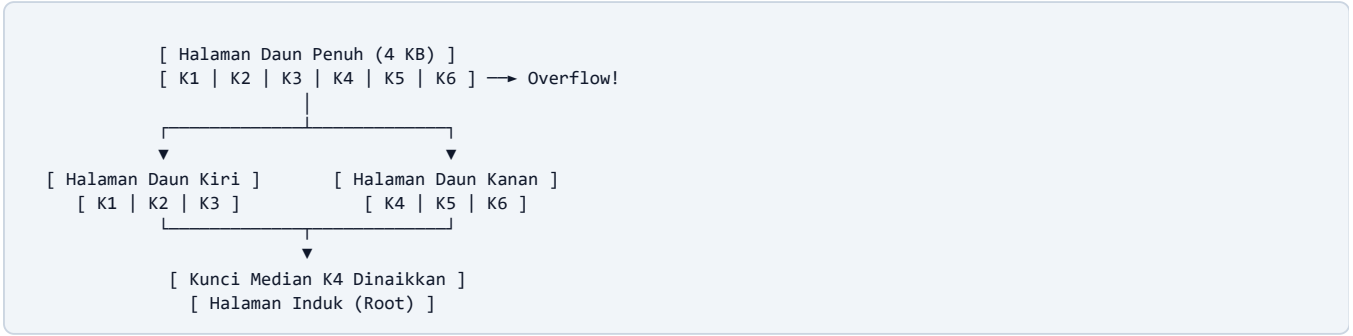
    untuk kasir_id dalam ["KASIR-01", "KASIR-02", "KASIR-03"] {
        misal val_a = clock_a[kasir_id];
        misal val_b = clock_b[kasir_id];
        jika val_a > val_b { ada_a_lebih_besar = benar; }
        jika val_b > val_a { ada_b_lebih_besar = benar; }
    }

    jika ada_a_lebih_besar dan (bukan ada_b_lebih_besar) { kembalikan "A_MENDOMINASI"; }
    jika ada_b_lebih_besar dan (bukan ada_a_lebih_besar) { kembalikan "B_MENDOMINASI"; }
    kembalikan "KONFLIK_SEJAJAR"; // Butuh penggabungan (merge) manual
}
```

11 Integrasi Basis Data Mandiri WidyaBase Skala Industri

11.1 Arsitektur Internal B-Tree & Pembelahan Halaman (Page Split)

Ketika jumlah baris data dalam satu halaman B-Tree melebihi kapasitas 4096 bita, WidyaBase melakukan pembelahan halaman (*page split*) secara atomik. Kunci median dinaikkan ke halaman induk (*parent page*) untuk mempertahankan kedalaman pohon tetap seimbang (*balanced tree depth* $O(\log N)$):



11.2 Kueri Manufaktur Rantai Pasok: Bill of Materials (BOM)

```
impor "./modul/widyabase.wya";

fungsi analisis_kebutuhan_komponen_bom() {
    misal db = BukaWidyaBase("pabrik_otomotif.wdb");

    // CTE Rekursif untuk Menghitung Seluruh Sub-Komponen Kendaraan Listrik
    misal sql_bom = "
        DENGAN REKURSIF rincian_bom SEBAGAI (
            PILIH id_part, nama_komponen, jumlah, 1 SEBAGAI tingkat_sub
            DARI komponen_rakitan
            DIMANA id_induk ADALAH NIHIL

            GABUNG_SEMUA

            PILIH k.id_part, k.nama_komponen, r.jumlah * k.jumlah, r.tingkat_sub + 1
            DARI komponen_rakitan AS k
            GABUNG rincian_bom AS r PADA k.id_induk = r.id_part
        )
        PILIH * DARI rincian_bom URUTKAN_BERDASARKAN tingkat_sub NAIK;
    ";

    misal daftar_komponen = db.kueri(sql_bom);
    untuk p dalam daftar_komponen {
        cetak("Tingkat:", p["tingkat_sub"], "| Part:", p["nama_komponen"], "| Total Qty:", p["jumlah"]);
    }
}
```

11.3 Kueri Geospasial: Pencarian Armada Taksi Terdekat

```
fungsi cari_armada_terdekat(lintang_penumpang, bujur_penumpang) {
    misal db = BukaWidyaBase("transportasi_online.wdb");

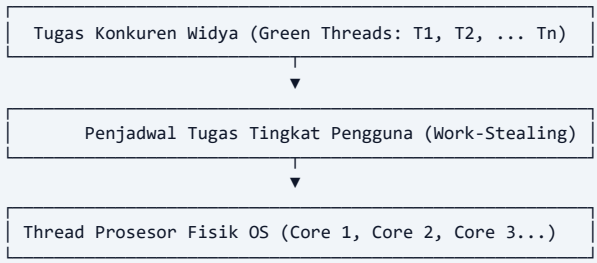
    // Menghitung jarak geodesik menggunakan fungsi geometris bawaan WidyaBase
    misal sql_jarak = "
        PILIH
            id_driver,
            nama_supir,
            ST_Jarak(lokasi_gps, ST_Titik(" + ke_teks(lintang_penumpang) + ", " + ke_teks(bujur_penumpang) + ")) AS radius_
        DARI armada_taksi
        DIMANA status_siaga = BENAR
        URUTKAN_BERDASARKAN radius_meter NAIK
        BATAS 3;
    ";

    misal supir_terpilih = db.kueri(sql_jarak);
    cetak("=== 3 ARMADA TERDEKAT DENGAN ANDA ===");
    untuk s dalam supir_terpilih {
        cetak("Supir:", s["nama_supir"], "| Jarak:", s["radius_meter"], "meter");
    }
}
```

12 Pemrograman Konkuren, Paralelisme & Jaringan Mikroservis

12.1 Green Threads & Task Worker: Model Konkurensi Ringan

Dalam arsitektur komputasi modern, pembuatan thread sistem operasi (*OS native thread*) memerlukan biaya alokasi memori yang cukup besar (umumnya 1–2 MB per tumpukan thread). Widya-Lang menerapkan model konkurensi **Green Threads (M:N Task Scheduler)** di mana ribuan tugas komputasi independen dijadwalkan secara kooperatif di atas sejumlah kecil thread fisik prosesor. Dengan jejak tumpukan awal hanya 4 KB, sistem dapat menjalankan 100.000 tugas konkuren secara simultan tanpa menghabiskan kapasitas RAM server.



12.2 Saluran Komunikasi (Channels) Antar-Tugas & Worker Pool

Pola **Worker Pool** adalah standar industri untuk membatasi konsumsi sumber daya saat memproses ribuan pekerjaan komputasi berat secara paralel:

```
impor "./modul/konkurensi.wya";

fungsi jalankan_worker_pool(jumlah_pekerja, daftar_tugas) {
    misal kanal_tugas = buat_saluran(panjang(daftar_tugas));
    misal kanal_hasil = buat_saluran(panjang(daftar_tugas));

    // Meluncurkan N pekerja (Worker Green Threads)
    untuk id_pekerja dalam 1..jumlah_pekerja {
        jalankan_tugas(fungsi() {
            selama benar {
                misal tugas = terima_dari_saluran(kanal_tugas);
                jika tugas == nihil { henti; }
                cetak("[Pekerja-" + ke_teks(id_pekerja) + "] Memproses:", tugas.id);
                misal hasil = tugas.nilai * 2; // Komputasi
                kirim_ke_saluran(kanal_hasil, { "id": tugas.id, "hasil": hasil });
            }
        });
    }

    // Menerbitkan tugas ke dalam antrean
    untuk t dalam daftar_tugas {
        kirim_ke_saluran(kanal_tugas, t);
    }
    tutup_saluran(kanal_tugas);
}
```

12.3 Middleware Otentikasi Token JWT pada Server HTTP

Mengamankan rute mikroservis dengan memvalidasi tanda tangan JSON Web Token (JWT) secara ketat pada lapisan perantara:

```
impor "./modul/http_server.wya";
impor "./modul/kripto_aman.wya";

fungsi middleware_jwt_auth(permintaan, respon, handler_berikutnya) {
    misal header_auth = permintaan.headers["Authorization"];
    jika header_auth == nihil atau bukan diawali_dengan(header_auth, "Bearer ") {
        respon.kirim_json(401, { "galat": "Akses Ditolak: Token Bearer Tidak Ditemukan!" });
        kembalikan;
    }

    misal token = potong_teks(header_auth, 7);
    coba {
        misal klaim = VerifikasiTokenJWT(token, "kunci_rahasia_otoritas_2026");
        permintaan.konteks_user = klaim;
        handler_berikutnya(permintaan, respon);
    } tangkap (err) {
        respon.kirim_json(403, { "galat": "Token Tidak Sah atau Telah Kedaluwarsa!" });
    }
}
```

13 Komputasi Sains, Vektor AI & Data Engineering

13.1 Aljabar Linier: Perkalian Matriks Berkinerja Tinggi

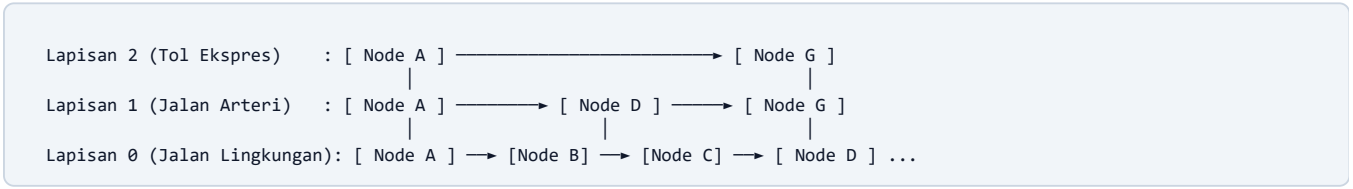
Operasi perkalian matriks adalah fondasi utama lapisan jaringan saraf tiruan (*deep learning neural networks*). Berikut adalah implementasi algoritma perkalian matriks murni di Widya-Lang:

```
fungsi perkalian_matriks(mat_a, mat_b) {
    misal baris_a = panjang(mat_a);
    misal kolom_a = panjang(mat_a[0]);
    misal kolom_b = panjang(mat_b[0]);

    misal hasil = [];
    untuk i dalam 0..(baris_a - 1) {
        misal baris_baru = [];
        untuk j dalam 0..(kolom_b - 1) {
            misal akumulasi = 0;
            untuk k dalam 0..(kolom_a - 1) {
                akumulasi = akumulasi + (mat_a[i][k] * mat_b[k][j]);
            }
            tambah(baris_baru, akumulasi);
        }
        tambah(hasil, baris_baru);
    }
    kembalikan hasil;
}
```

13.2 Navigasi Graf Multi-Lapisan HNSW (Hierarchical Navigable Small World)

Pencarian vektor HNSW mengorganisasikan data ke dalam lapisan graf bertingkat layaknya jaringan jalan tol ekspres dan jalan raya perkotaan:



14 Kolaborasi Cerdas dengan AI Lokal (WidyaCoder)

14.1 Mengoperasikan WidyaCoder di Terminal Lokal

```
# Menjalankan model AI lokal yang telah dilatih korpus Widya-Lang
$ ollama run widya-coder:latest
>>> Bagaimana cara membuat tabel di WidyaBase dengan kunci utama otomatis?

Kompiler WidyaBase menyediakan sintaksis DDL Dialek Nusantara:
BUAT_TABEL pengguna (
    id INT KUNCI_UTAMA,
    nama_lengkap TEKS,
    saldo INT
);
```

14.2 Siklus Agen AI Otonom Multi-Putaran (Multi-Turn ReAct Loop)

```
impor "./modul/http_klien.wya";
impor "./modul/widyabase.wya";

fungsi agen_cerdas_investigasi(pertanyaan_pengguna) {
    misal db = BukaWidyaBase("operasional.wdb");
    misal memori_percakapan = [
        { "role": "system", "content": "Anda adalah Analis Sistem Widya-Lang berdaulat. Anda dapat mengeksekusi kueri Widya
    ];

    tambah(memori_percakapan, { "role": "user", "content": pertanyaan_pengguna });

    // Putaran 1: Minta AI Menganalisis Kebutuhan Data
    misal respon_ai = PanggilOllamaChat("widya-coder:latest", memori_percakapan);
    cetak("[AI PIKIRAN]:", respon_ai);

    // Deteksi Pemanggilan Alat (Tool Calling)
    jika mengandung_teks(respon_ai, "[KUERI:") {
        misal perintah_sql = ekstrak_antara(respon_ai, "[KUERI:", "]");
        cetak("[AKSI EKSEKUSI WIDYABASE]:", perintah_sql);

        misal data_tabel = db.kueri(perintah_sql);
        tambah(memori_percakapan, { "role": "assistant", "content": respon_ai });
        tambah(memori_percakapan, { "role": "user", "content": "HASIL DATA WIDYABASE:\n" + ke_json(data_tabel) });

        // Putaran 2: AI Menyusun Jawaban Akhir Berdasarkan Data Riil
        misal kesimpulan_akhir = PanggilOllamaChat("widya-coder:latest", memori_percakapan);
        cetak("[KESIMPULAN FINAL AGEN]:\n", kesimpulan_akhir);
        kembalikan kesimpulan_akhir;
    }

    kembalikan respon_ai;
}
```

15 Keamanan Siber, Kriptografi & Kepatuhan UU PDP

15.1 Kriptografi Pasca-Kuantum (Post-Quantum Cryptography: ML-KEM / Kyber-768)

Perkembangan komputer kuantum di masa depan berpotensi meruntuhkan sistem enkripsi RSA dan kurva eliptis klasik. Widya-Lang adalah salah satu pelopor bahasa berdaulat yang telah mengintegrasikan modul pertukaran kunci berbasis kekisi (*lattice-based cryptography*) standar NIST **ML-KEM (Kyber-768)** di dalam `modul/kripto_aman.wya`:

```
impor "./modul/kripto_aman.wya";

fungsi pertukaran_kunci_kuantum() {
    // 1. Penerima (Server Bank) Membangkitkan Pasangan Kunci Kyber-768
    misal kunci_server = BuatPasanganKunciKyber768();
    cetak("Kunci Publik Kuantum Server (1184 Bita):", panjang(kunci_server.publik));

    // 2. Pengirim (Aplikasi Nasabah) Melakukan Enkapsulasi Kunci Bersama
    misal enkapsulasi = EnkapsulasiKyber768(kunci_server.publik);
    // enkapsulasi menghasilkan: ciphertext (1088 bita) dan kunci_bersama (32 bita)

    // 3. Penerima Melakukan Dekapsulasi Kunci Bersama dengan Kunci Privat
    misal kunci_terdekapsulasi = DekapsulasiKyber768(enkapsulasi.ciphertext, kunci_server.privat);

    // Verifikasi kedua pihak memiliki kunci rahasia simetris yang identik
    misal aman = perbandingan_waktu_konstan(enkapsulasi.kunci_bersama, kunci_terdekapsulasi);
    cetak("Kunci Bersama Kuantum Sukses Disepakati:", aman); // benar
}
```

15.2 Perbandingan Waktu Konstan (Constant-Time Comparison)

Serangan waktu (*timing attacks*) memanfaatkan perbedaan mikro-detik dalam pembandingan string untuk menebak sandi atau token. Widya-Lang menyediakan fungsi pembandingan yang kebal terhadap kebocoran waktu komputasi:

```
fungsi perbandingan_waktu_konstan(teks_a, teks_b) {
    jika panjang(teks_a) != panjang(teks_b) {
        kembalikan salah;
    }
    misal akumulasi_xor = 0;
    misal n = panjang(teks_a);
    untuk i dalam 0..(n - 1) {
        akumulasi_xor = akumulasi_xor | (kode_karakter(teks_a, i) ^ kode_karakter(teks_b, i));
    }
    kembalikan akumulasi_xor == 0;
}
```

15.3 Daftar Periksa Audit Kepatuhan UU PDP No. 27/2022

Pasal Regulasi	Kewajiban Pengendali Data Pribadi	Mekanisme Implementasi di Widya-Lang
Pasal 20	Memiliki dasar pemrosesan data pribadi yang sah & eksplisit	Pencatatan field `persetujuan_pengguna_ts` saat registrasi entitas.
Pasal 35	Melindungi keamanan data dari pemrosesan tanpa hak	Enkripsi kolom database sensitif menggunakan AES-256-GCM.
Pasal 37	Mencegah akses yang tidak sah melalui kontrol ketat	Autentikasi JWT Bearer bertanda tangan Ed25519 & Role-Based Access Control.
Pasal 43	Menghapus data pribadi jika masa retensi telah berakhir	Kueri terjadwal `HAPUS_PERMANEN DARI pelanggan DIMANA kedaluwarsa < SEKARANG()` dengan penghapusan memori kriptografis nol.

16 Praktik Pengujian Perangkat Lunak, Benchmarking & Standar Koding

16.1 Pengujian Berbasis Properti (Property-Based Testing)

Selain pengujian kasus unit statis, pengujian berbasis properti menguji apakah invarian logika matematis tetap berlaku di bawah ribuan masukan acak:

```
impor "./modul/pengujian.wya";

tes_properti "Invarian: Komutatif Penjumlahan Selalu Berlaku" {
    uji_untuk_semua(tipe_data_acak("angka"), tipe_data_acak("angka"), fungsi(a, b) {
        pastikan_sama(a + b, b + a);
    });
}

tes_properti "Invarian: Enkripsi yang Diikuti Dekripsi Mengembalikan Nilai Asli" {
    uji_untuk_semua(tipe_data_acak("teks"), fungsi(pesan_asli) {
        misal sandi = EnkripsiAES256GCM(pesan_asli, KUNCI_UJI);
        misal balik = DekripsiAES256GCM(sandi.ciphertext, KUNCI_UJI, sandi.nonce, sandi.tag);
        pastikan_sama(pesan_asli, balik);
    });
}
```

17 Panduan Distribusi, Kompilasi Biner & DevOps

17.1 Konfigurasi Unit Layanan Linux (systemd service)

```
[Unit]
Description=Layanan Server Mikroservis Widya-Lang Nusantara
After=network.target

[Service]
Type=simple
User=widya
WorkingDirectory=/opt/widya_app
ExecStart=/usr/local/bin/widya jalan aplikasi_server.wya
Restart=always
RestartSec=5s
LimitNOFILE=65535
Environment="WIDYA_ENV=produksi"
Environment="PORT=8080"

[Install]
WantedBy=multi-user.target
```

17.2 Metrik Operasional Prometheus Exporter

Widya-Lang menyediakan endpoint metrik standar Prometheus pada `/metrik` untuk monitoring performa CPU, memori heap, dan latensi query secara realtime melalui dashboard Grafana:

```
impor "./modul/metrik.wya";

fungsi ekspos_metrik_prometheus(server) {
    misal counter_transaksi = BuatPenghitungMetrik("transaksi_total", "Jumlah transaksi kasir berhasil");

    server.rute_get("/metrik", fungsi(req, res) {
        misal data_metrik = KumpulkanMetrikSistem();
        res.kirim_teks(200, data_metrik, "text/plain; version=0.0.4");
    });
}
```

A Lampiran: Kamus Lengkap Sintaks Dialek Nusantara

Kata Kunci Nusantara	Padanan Bahasa Inggris	Kategori Sintaksis	Deskripsi Semantik & Contoh Penggunaan
`misal`	`let / var / mut`	Variabel	Mendeklarasikan variabel mutable dinamis: `misal x = 10;`
`tetap` / `tetapan`	`const / immutable`	Konstanta	Mendeklarasikan konstanta terkunci: `tetap PI = 3.14;`
`fungsi`	`function / def / fn`	Subrutin	Mendefinisikan blok fungsi komputasi: `fungsi hitung() { }`
`kembalikan`	`return`	Alur Fungsi	Mengembalikan nilai dari eksekusi fungsi: `kembalikan total;`
`jika`	`if`	Percabangan	Mengevaluasi kondisi logika pertama: `jika nilai >= 75 { }`
`kalau`	`else if / elif`	Percabangan	Mengevaluasi cabang kondisi berikutnya: `kalau nilai >= 60 { }`
`lainnya`	`else`	Percabangan	Cabang alternatif jika seluruh kondisi salah: `lainnya { }`
`untuk`	`for`	Perulangan	Iterasi rentang angka atau koleksi data: `untuk i dalam 1..10 { }`
`selama`	`while`	Perulangan	Perulangan berbasis syarat kondisi benar: `selama aktif { }`
`henti`	`break`	Kendali Loop	Menghentikan perulangan seketika: `henti;`
`lanjut`	`continue`	Kendali Loop	Melompati iterasi saat ini menuju iterasi berikutnya: `lanjut;`
`struktur`	`struct / class`	Tipe Bentukan	Mendefinisikan entitas data terstruktur: `struktur Nasabah { }`
`sifat`	`trait / interface`	Kontrak Perilaku	Mendefinisikan antarmuka polimorfisme: `sifat Transaksi { }`
`coba`	`try`	Penanganan Galat	Membuka blok eksekusi rawan kegagalan: `coba { ... }`
`tangkap`	`catch / except`	Penanganan Galat	Menangkap objek galat yang terlempar: `tangkap (galat) { ... }`
`akhirnya`	`finally`	Penanganan Galat	Blok yang dijamin selalu dieksekusi: `akhirnya { tutup(); }`
`lempar`	`throw / raise`	Pemicu Galat	Memicu galat terstruktur secara eksplisit: `lempar "Saldo minus";`
`impor`	`import / use`	Modularitas	Memuat berkas atau pustaka lain: `impor "./modul/desktop.wya";`
`cetak`	`print / println`	I/O Standar	Mencetak baris data ke terminal konsol: `cetak("Halo Indonesia");`
`dan`	`&& / and`	Operator Logika	Evaluasi konjungsi logika biner: `jika a dan b { }`
`atau`	`|| / or`	Operator Logika	Evaluasi disjungsi logika biner: `jika a atau b { }`
`bukan`	`! / not`	Operator Logika	Negasi nilai kebenaran logika: `jika bukan aktif { }`
`benar` / `salah`	`true / false`	Literal Boolean	Konstanta nilai kebenaran logika: `misal valid = benar;`
`nihil`	`null / nil / none`	Literal Ketiadaan	Menandakan ketiadaan nilai: `misal data = nihil;`
`panjang`	`len / length / size`	Fungsi Bawaan	Menghitung panjang deret atau jumlah karakter string: `panjang(deret)`
`tambah`	`push / append`	Fungsi Bawaan	Menyisipkan elemen ke bagian ekor deret: `tambah(daftar, item)`
`iris`	`slice / substring`	Fungsi Bawaan	Mengambil potongan deret atau string: `iris(teks, 0, 5)`
`ke_angka`	`parseInt / toInt`	Konversi Tipe	Mengonversi representasi string menjadi integer: `ke_angka("150")`
`ke_teks`	`toString / str`	Konversi Tipe	Mengonversi nilai tipe apapun menjadi string teks: `ke_teks(42)`

B Lampiran: Tabel Komparasi Bahasa Lintas Platform

Fitur / Karakteristik	Widya-Lang v1.0	Rust 1.80	Python 3.12	Go 1.22	TypeScript / Node
Dialek Bahasa Asli	Bahasa Indonesia (Nusantara)	Bahasa Inggris	Bahasa Inggris	Bahasa Inggris	Bahasa Inggris
Mesin Komputasi	Native Rust VM & Compiler	LLVM Native Binary	CPython Bytecode VM	Go SSA Native Compiler	V8 JIT Engine
Jejak Startup Biner	< 5 ms (Instan)	< 2 ms (Instan)	~ 50-100 ms	< 5 ms (Instan)	~ 150-250 ms
Basis Data Tersemat	WidyaBase 16 Pilar Bawaan	Membutuhkan Crate Luar	SQLite Bawaan	Membutuhkan Driver Cgo	Membutuhkan Paket NPM
Dukungan GUI Native	Tersedia (modul/desktop.wya)	Pustaka Eksternal Rumit	Tkinter Lambat	Eksternal Cgo	Electron Berat (>150MB)
Dukungan Model AI Lokal	WidyaCoder (Ollama Native)	Candle / Burn bindings	PyTorch / Transformers	Ollama REST client	LangChain / Ollama JS
Standar Finansial Nasional	QRIS ASPI EMVCo Terintegrasi	Tidak Ada Standar	Tidak Ada Standar	Tidak Ada Standar	Tidak Ada Standar

C Lampiran: Direktori Modul Ekosistem Resmi

Path Berkas Modul Resmi	Fungsi Utama & Tanggung Jawab Komputasi	Contoh Penggunaan API Kunci
`modul/widyabase.wya`	Mesin basis data relasional slotted-page 4KB, transaksi ACID, WAL, CTE rekursif, dan window functions.	`misal db = BukaWidyaBase("data.wdb");`
`modul/desktop.wya`	Framework antarmuka grafis desktop native ringan (Windows/Linux) untuk aplikasi kasir & dashboard.	`misal app = AplikasiDesktop("Kasir", 800, 600);`
`modul/mobile.wya`	Framework mobile sentuh dengan pembacaan kamera QRIS ASPI, printer bluetooth, dan outbox offline.	`misal mob = AplikasiMobile("Warung Mobile");`
`modul/dataframe_analitika.wya`	Struktur data tabel kolumnar bervolume tinggi, operasi pembersihan missing values, agregasi GroupBy.	`misal df = MuatDataframeCSV("omzet.csv");`
`modul/vektor_hnsw.wya`	Indeks pencarian semantik vektor kemiripan kosinus untuk arsitektur Retrieval-Augmented Generation (RAG).	`misal idx = IndeksVektorHNSW(128);`
`modul/kripto_aman.wya`	Kriptografi kelas militer: AES-256-GCM, kurva Ed25519, hashing Argon2id, dan Post-Quantum Kyber.	`misal enc = EnkripsiAES256GCM(teks, kunci);`
`modul/http_server.wya`	Server HTTP mikroservis berkecepatan tinggi dengan router RESTful dan penangan JSON otomatis.	`misal srv = BuatServerHTTP("0.0.0.0", 8080);`

D Lampiran: Diagnostik Galat Compiler & Solusi Perbaikan

Kode Galat	Pesan Diagnostik Kompiler	Akar Penyebab Permasalahan	Tindakan Solusi Pemulihan Kode
W001	Tanda titik koma (`;`) tidak ditemukan di akhir baris	Lupa menutup pernyataan perintah komputasi	Tambahkan karakter `;` pada akhir baris instruksi.
W002	Simbol pengenal variabel belum dideklarasikan	Mengakses variabel sebelum dideklarasikan dengan `misal`	Pastikan mendeklarasikan variabel dengan `misal nama = nilai;` sebelum diakses.
W004	Upaya mutasi pada tetapan abadi terlarang	Mencoba mengubah nilai variabel yang dideklarasikan dengan `tetap`	Ganti kata kunci deklarasi menjadi `misal` jika nilai memang perlu berubah.
W008	Indeks deret berada di luar batas alokasi memori	Mengakses elemen deret dengan indeks >= panjang deret	Periksa panjang koleksi dengan fungsi `panjang(deret)` sebelum mengakses indeks.
W015	Kueri WidyaBase melanggar kendala kunci asing	Menyisipkan ID relasi yang tidak terdaftar di tabel induk	Pastikan entitas pada tabel rujukan telah terdaftar sebelum menyisipkan baris anak.
W021	Checksum CRC-16 QRIS tidak sesuai dengan payload	String QRIS mengalami korupsi atau manipulasi data saat pemindaian	Minta pembeli melakukan scan ulang kode QRIS dari aplikasi perbankan resmi.

ID Penutup: Menyongsong Masa Depan Teknologi Berdaulat

Dengan selesainya buku panduan resmi jilid 2 ini, Anda telah menguasai seluruh spektrum rekayasa perangkat lunak modern: dari algoritma dasar Nusantara, rekayasa desktop kasir UMKM, transaksi pembayaran QRIS nasional, basis data mandiri WidyaBase, hingga orkestrasi agen AI lokal tanpa ketergantungan asing. Selamat berkarya, berinovasi, dan mengibarkan kedaulatan teknologi Indonesia Emas 2045!